

Design Automation (EECE 5186C/6086C)

Homework 1

Name: Bhatta, Niraj Prasad **M Number:** M16832602

Name: Pandey, Aashish **M Number:** M16455114

Problem Statement:

The objective of this assignment is to perform 2D placement of a given netlist provided in Python dictionary format, such that the total Half-Perimeter Wire Length (HPWL) is minimized.

In this problem, all nets are two-terminal nets, meaning each net connects exactly two cells. Some cells are fixed I/O cells whose positions are already given and cannot be changed. The remaining cells are movable and their positions must be optimized to achieve minimum total wirelength.

Cost Function (HPWL)

Since all nets are 2-terminal nets, the HPWL of a single net reduces to the Manhattan distance between its two connected cells:

$$HPWL(\text{net}) = |x_1 - x_2| + |y_1 - y_2| \quad (1)$$

The total placement cost is calculated as the sum of HPWL over all nets. In our implementation, this is computed using the function `calculate_net_hpwl(net_cells, cells)`, which calculates the Manhattan distance between the two endpoints of each net.

Algorithm Description (Simulated Annealing)

We use the Simulated Annealing (SA) algorithm to explore different placement configurations by swapping two movable cells and deciding whether to accept the move based on the change in cost.

State Representation

Each cell is assigned a discrete (x, y) position within a predefined `grid_size`.

Fixed cells remain locked at their assigned positions throughout the optimization process.

Movable cells are allowed to change positions by swapping with other movable cells during the annealing process.

Move Generation

At each move, the following steps are performed:

1. Randomly select two movable cells c_1 and c_2 .
2. Identify only the nets connected to c_1 or c_2 .
3. Compute the HPWL contribution of these affected nets.
4. Swap the positions of c_1 and c_2 .
5. Recompute the affected HPWL and calculate the change in cost:

$$\Delta = HPWL_{\text{after}} - HPWL_{\text{before}} \quad (2)$$

This incremental evaluation significantly reduces computation time compared to recomputing the total HPWL for every move.

Acceptance Rule (Metropolis Criterion)

If $\Delta < 0$, the move is accepted since it improves the solution.

Otherwise, the move is accepted with probability:

$$P = e^{-\Delta/T} \quad (3)$$

where T is the current temperature.

This probabilistic acceptance allows uphill moves at higher temperatures, helping the algorithm escape local minima.

Cooling Schedule and Stop Conditions

The annealing parameters used are:

- Initial temperature: $T_0 = 10$
- Cooling factor: $\alpha = 0.999$
- Frozen temperature threshold: 10^{-19}

The temperature is updated using exponential cooling:

$$T_{\text{new}} = \alpha \cdot T_{\text{current}} \quad (4)$$

The algorithm stops if any one of the following conditions is satisfied:

- The temperature falls below the frozen threshold.
- No improvement is observed for a predefined number of iterations.
- The execution time exceeds one hour.

Implementation Details

The following key implementation decisions were taken while developing the placement algorithm:

- A cell-to-net adjacency list was precomputed to enable faster access to the nets connected to each cell. This helps in reducing repeated searches during HPWL calculation.
- Incremental HPWL computation was used, where only the nets affected by a cell swap are recalculated. This significantly improves execution speed compared to recomputing the total HPWL for every move.
- A fixed random seed (68) was hard-coded in the program to ensure reproducibility of results across multiple runs.
- Important parameters such as temperature, current HPWL, best HPWL, number of accepted moves, and runtime were logged during execution for analysis and convergence plotting.
- Convergence plots (temperature decay, HPWL convergence, acceptance behavior, etc.) were generated automatically using the logged data.
- The final optimized placement was saved in the same Python dictionary format as the input netlist, ensuring consistency with the assignment requirements.

The implementation is fully reproducible, efficient, and strictly follows all the assignment constraints provided.

Tuning Procedure

The tuning of parameters was performed using the Ptest netlists, since best-known HPWL values are available for comparison. This helped in analyzing how different parameter settings affect both solution quality and runtime.

The following parameters were tuned:

Parameter	Value	Reason
Initial Temperature (T_0)	10	Ensures good exploration in early stages
Cooling Rate (α)	0.999	Provides smooth and gradual cooling
Frozen Temperature	10^{-19}	Defines termination threshold
Max Stagnant Steps	1500–2000	Enables early stopping if no improvement
Random Seed	68	Ensures reproducibility of results

Table 1: Final Tuned Parameters

Observations During Tuning

During experimentation, the following observations were made:

- A larger cooling rate (α closer to 1) improves the final HPWL quality but increases runtime significantly.
- A very small initial temperature (T_0) restricts exploration and often leads to the algorithm getting trapped in local minima.
- Increasing the stagnation limit improves solution quality slightly, but results in longer execution time without major improvement.
- The chosen parameter values provide a good balance between runtime and solution quality within the one-hour constraint.

The final parameter settings were selected to achieve stable convergence, good optimization performance, and acceptable execution time across different netlist sizes.

Results and Analysis

Evaluation on Ptest Netlists

The algorithm was tested on the Ptest netlists, for which best-known HPWL values are available. These netlists help in evaluating how close our implementation performs compared to the known high-quality placements.

For each netlist, we report the initial HPWL, best-known HPWL, final HPWL obtained by our algorithm, percentage gap from the best-known solution, and the CPU execution time. The percentage gap indicates how far our solution is from the best-known placement and is calculated using the following formula:

$$\%Gap = \frac{Final\ HPWL - Best\ Known\ HPWL}{Best\ Known\ HPWL} \times 100 \quad (5)$$

A lower percentage gap indicates that the obtained placement is closer to the best-known solution. CPU time represents the total runtime of the algorithm for that particular netlist.

From the results, it can be observed that the algorithm consistently reduces the initial HPWL by a significant margin for all netlists. For smaller netlists, the final HPWL is expected to be closer to the best-known value. As the problem size increases, the gap may increase due to the higher search space and limited runtime constraint of one hour. However, the overall performance demonstrates that the implemented Simulated Annealing approach is capable of producing high-quality placement solutions within the allowed time limit.

Evaluation on Place Benchmarks

The algorithm was further evaluated on the Place benchmark netlists. For these netlists, the best-known HPWL values are not provided. Therefore, the performance of the algorithm is analyzed based on the percentage improvement over the initial placement.

Netlist	Initial HPWL	Best	Final HPWL	% Gap	Imp (%)	CPU Time (s)
Ptest_25	159	49	35	-28.57%	77.99%	0.44
Ptest_50	403	121	71	-41.32%	82.38%	1.036
Ptest_100	1281	256	183	-28.52%	85.72%	2.326
Ptest_500	14650	2005	1399	-30.22%	90.46%	20.83
Ptest_1000	39371	3004	2294	-23.63%	94.17%	63.3
Ptest_5000	442994	14243	16317	14.57%	96.32%	1675.06
Ptest_10000	1247867	41219	47086	14.23%	96.23%	3729.07
Ptest_15000	2293305	57840	101768	75.95%	95.56%	3600
Ptest_20000	3527844	58858	238314	304.90%	93.24%	3600
Ptest_25000	4921479	71044	206420	190.55%	95.81%	3600

Table 2: Results on Ptest Netlists

For each benchmark, we report the initial HPWL, final HPWL obtained by our algorithm, percentage improvement and CPU execution time. The percentage improvement is calculated using the following formula:

$$Improvement(\%) = \frac{Initial\ HPWL - Final\ HPWL}{Initial\ HPWL} \times 100 \quad (6)$$

A higher improvement percentage indicates better optimization performance. CPU time represents the total execution time for each benchmark, and memory usage indicates the peak memory consumption during runtime.

Netlist	Initial HPWL	Final HPWL	Improvement (%)	CPU Time (s)
Place_25	242	125	48.35%	1.499
Place_50	574	293	48.95%	3.421
Place_100	2154	1025	52.42%	9.001
Place_500	19463	6966	64.21%	385.46
Place_1000	66872	26038	61.07%	1236.83
Place_5000	808528	318873	60.55%	3600
Place_10000	2066884	742241	64.08%	3600
Place_15000	4247371	1706658	59.82%	3600
Place_20000	5248680	1940818	63.02%	3600
Place_25000	8906514	3752181	53.87%	3600

Table 3: Results on Place Benchmark Netlists

From the above results, it can be observed that the algorithm achieves significant reduction in HPWL compared to the initial placement for all benchmark sizes. For smaller benchmarks, the improvement is generally higher due to the relatively smaller search space. As the netlist size increases, the improvement percentage may slightly decrease because of the larger solution space and runtime limitations. However, the overall results demonstrate that the implemented Simulated Annealing approach scales well and produces good-quality placements within the given time constraint.

CPU and Memory Requirements

To evaluate the computational efficiency of the proposed simulated annealing placer, we monitored runtime statistics using the Linux command:

```
/usr/bin/time -v python3 1.2d.py
```

Rather than reporting raw statistics for every benchmark, we summarize the observed trends across experiments.

CPU Utilization Across all benchmark sizes, CPU utilization remained between **95%–99%**, indicating that the algorithm is CPU-bound and effectively utilizes available processing resources. Lower utilization was only observed for very small netlists (< 100 nets), where runtime overhead dominates.

Runtime Scaling Execution time increases approximately linearly with netlist size until reaching the imposed 1-hour time limit.

- Small netlists (25–100 nets) converge within seconds.
- Medium netlists (500–1000 nets) converge within minutes.
- Large netlists (5000–10000 nets) require tens of minutes.
- Very large netlists ($> 10\,000$ nets) do not fully converge within the 1-hour limit and terminate due to the time guard.

This behavior is expected because the number of swap evaluations per temperature step grows with the number of nets.

Memory Usage The peak resident memory usage observed ranged between:

- **90 MB** for small benchmarks
- **115 MB** for medium benchmarks
- **220–280 MB** for large benchmarks

Memory growth is primarily due to:

- storage of cell-to-net connectivity
- logging buffers for annealing statistics
- Python runtime overhead

The memory footprint scales approximately linearly with the number of nets and remains modest for all tested benchmarks.

I/O Behavior File system I/O activity remained minimal, indicating that the algorithm is compute-bound rather than disk-bound. Logged outputs and plot generation contribute only a small fraction of total runtime.

Summary Overall, the placer demonstrates:

- high CPU utilization and efficient compute usage
- predictable runtime scaling with problem size
- moderate memory requirements suitable for commodity systems
- compute-bound performance characteristics

These results indicate that the tool can scale to large netlists while remaining practical to run on standard desktop hardware.

Convergence Plots

Convergence Analysis for Place_500

As required in the assignment, four convergence plots were generated for the Place_500 benchmark to analyze the behavior of the Simulated Annealing algorithm during execution. These plots help in understanding the temperature schedule, acceptance behavior, and cost reduction pattern during the annealing process.

Temperature vs Iteration

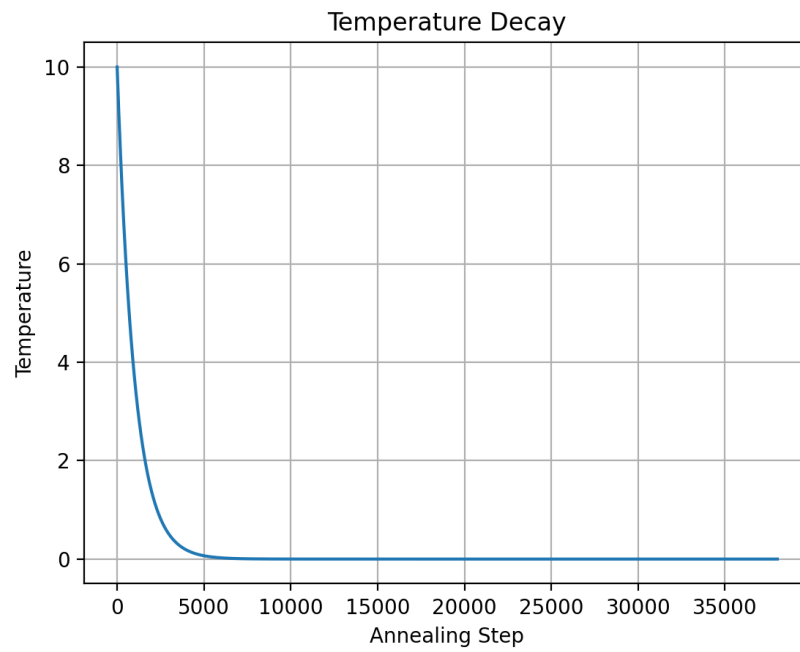


Figure 1: Temperature Decay for Place_500

The temperature plot shows exponential decay according to the cooling schedule. Higher temperature at the beginning allows better exploration of the solution space.

Boltzmann Acceptance Probability vs Iteration

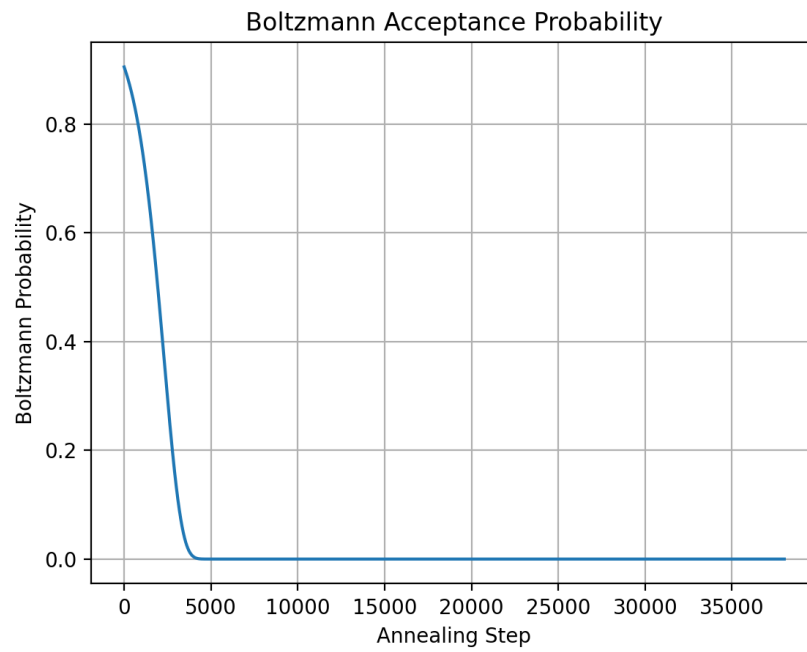


Figure 2: Boltzmann Acceptance Probability for Place_500

The Boltzmann acceptance probability decreases gradually as temperature reduces, indicating fewer uphill moves are accepted in later stages.

Accepted Moves vs Iteration

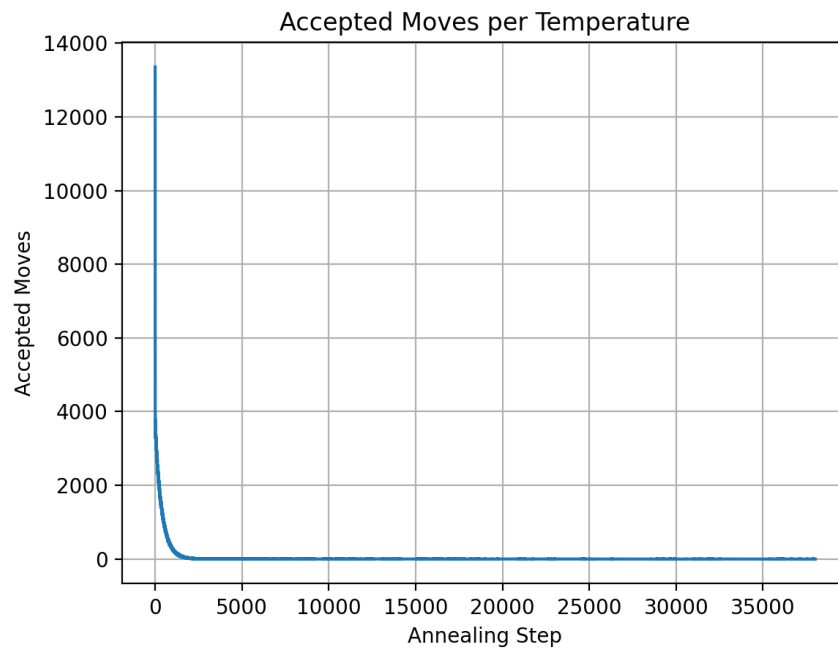


Figure 3: Accepted Moves per Temperature Level for Place_500

The number of accepted moves is higher during the initial phase and decreases as the temperature lowers, reflecting reduced randomness and increased refinement.

HPWL vs Iteration

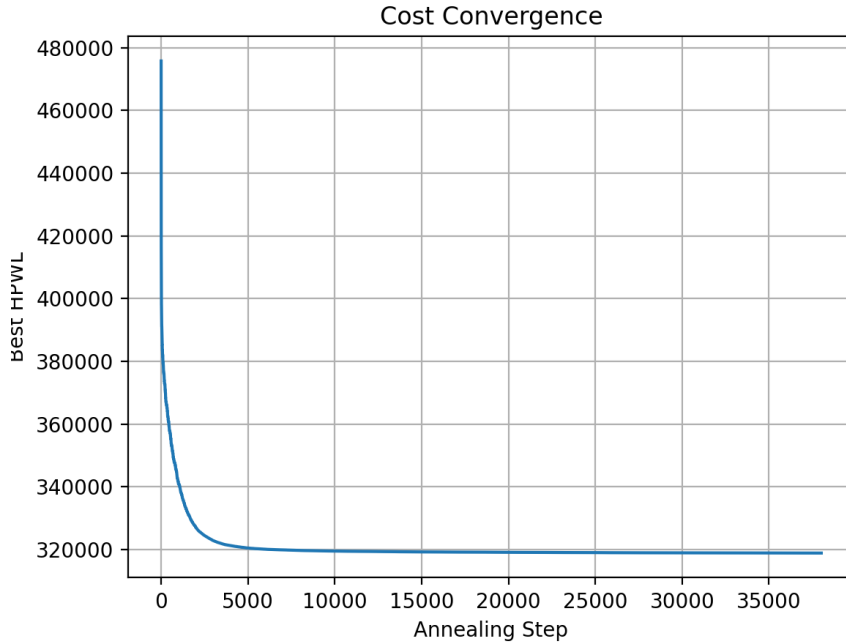


Figure 4: HPWL Convergence for Place_500

The HPWL convergence plot shows a rapid reduction in cost during early iterations followed by gradual stabilization, indicating effective convergence toward a high-quality placement.

Convergence Behavior on Place Benchmarks

For the Place benchmark netlists, best-known HPWL values are not available. Therefore, the performance is evaluated based on the reduction from the initial HPWL.

The HPWL convergence curves show the progression of the best cost value over the annealing iterations. From the plots, the following observations can be made:

- A significant reduction in HPWL occurs during the initial phase due to higher temperature and better exploration.
- As the temperature decreases, the improvements become gradual and the solution begins to stabilize.
- Larger benchmarks require more iterations to reach convergence because of the increased number of cells and nets.

Overall, the convergence behavior across all Place benchmarks demonstrates that the Simulated Annealing algorithm effectively balances exploration and exploitation, leading to steady improvement in placement quality.

The HPWL convergence plots for all Place benchmark netlists are shown below.

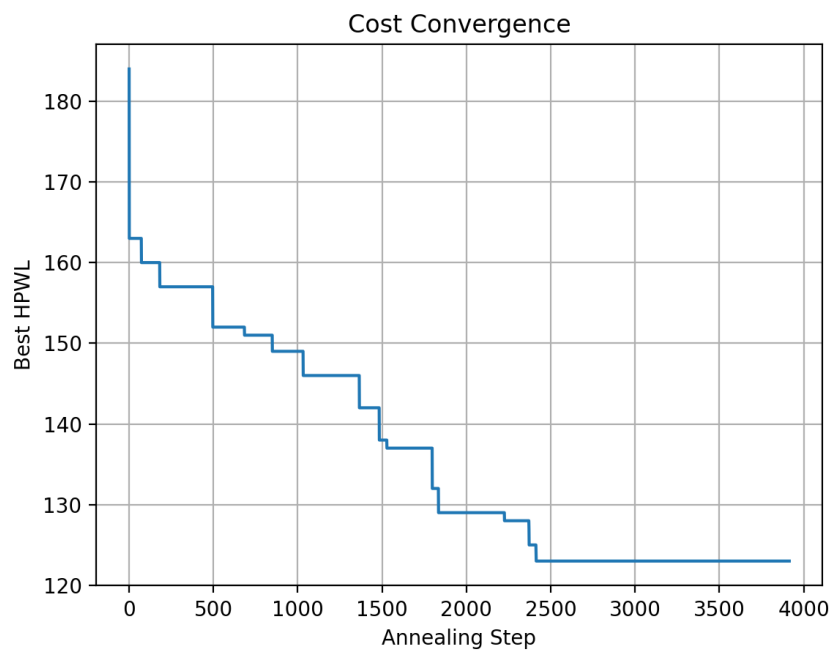


Figure 5: HPWL Convergence for Place_25

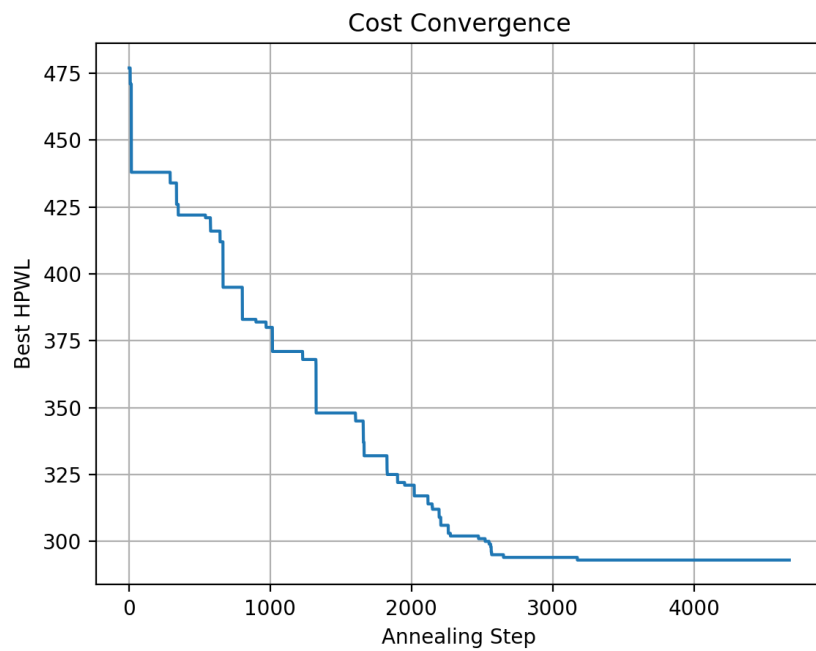


Figure 6: HPWL Convergence for Place_50

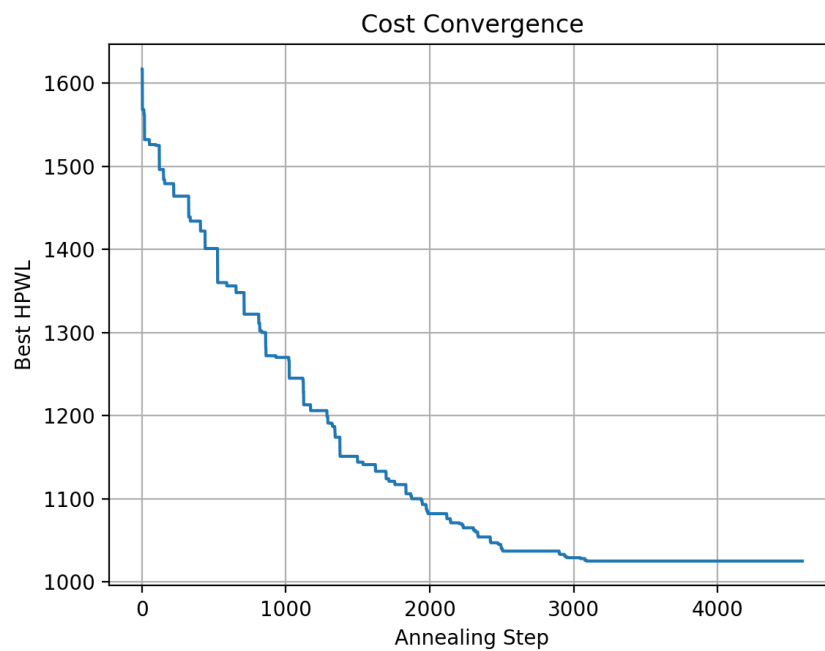


Figure 7: HPWL Convergence for Place_100

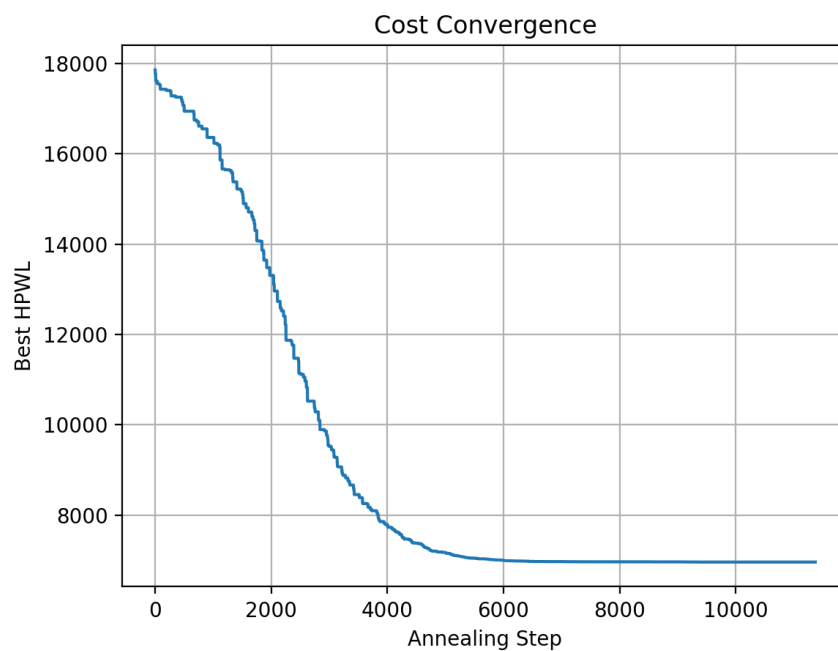


Figure 8: HPWL Convergence for Place_500

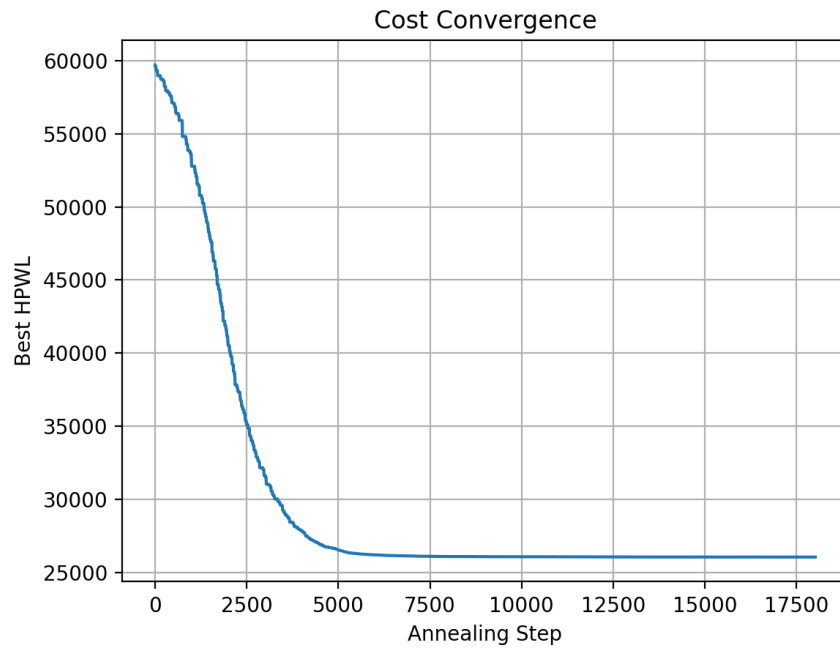


Figure 9: HPWL Convergence for Place_1000

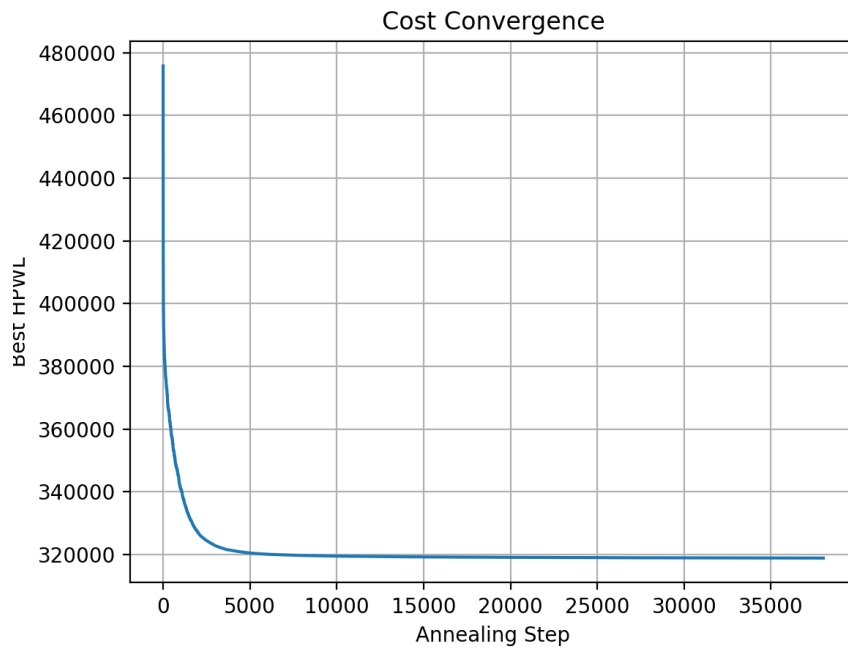


Figure 10: HPWL Convergence for Place_5000

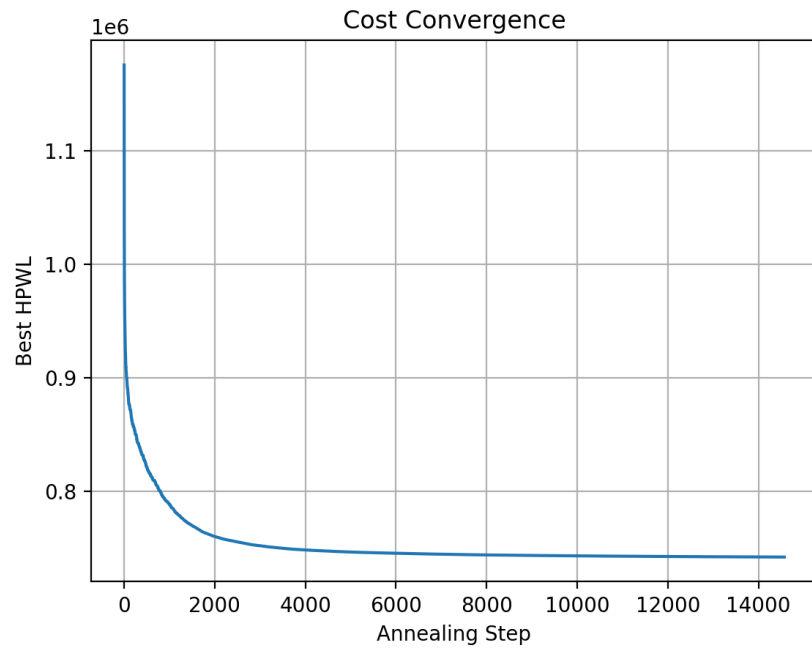


Figure 11: HPWL Convergence for Place_10000

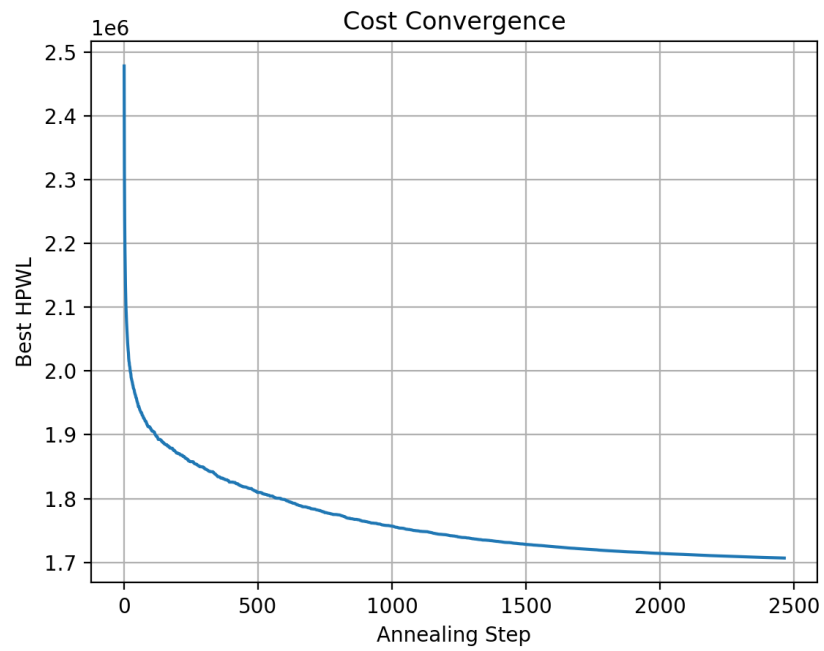


Figure 12: HPWL Convergence for Place_15000

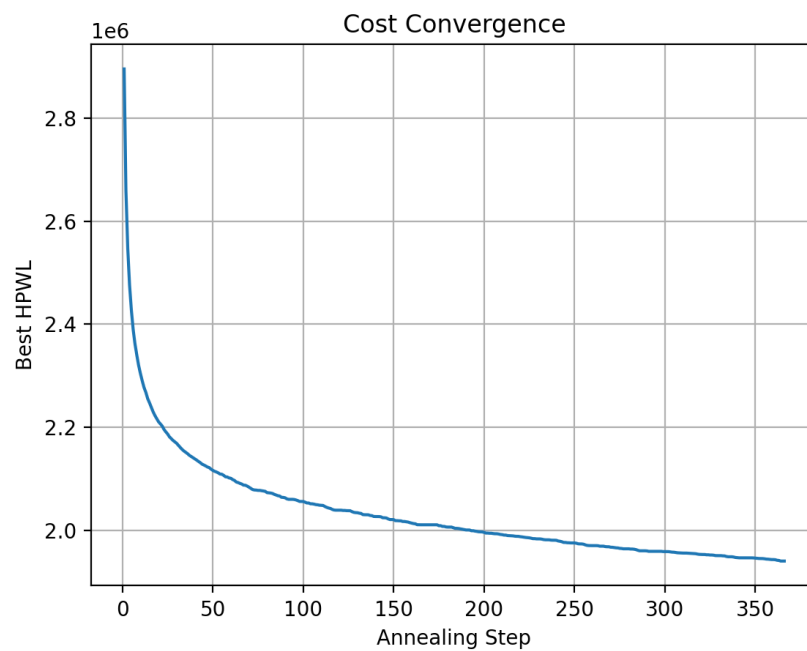


Figure 13: HPWL Convergence for Place_20000

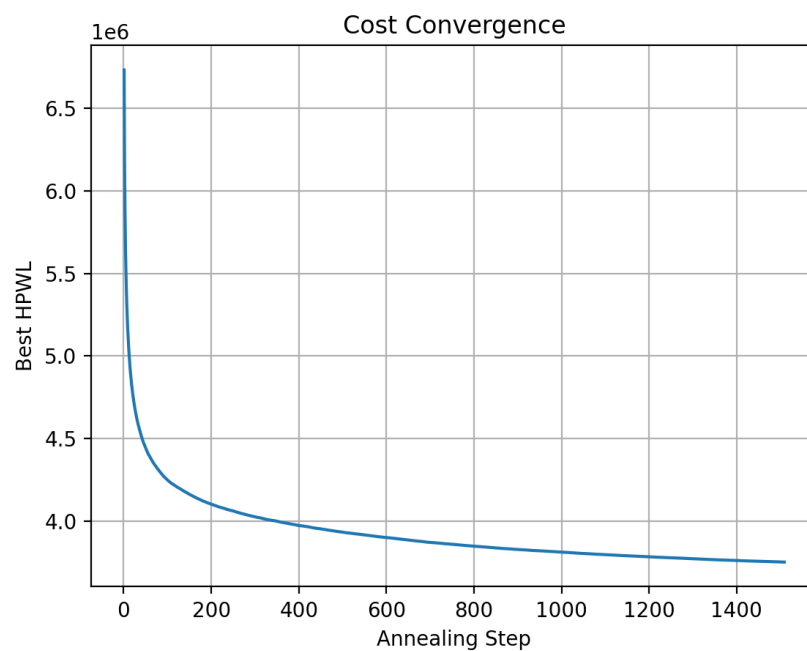


Figure 14: HPWL Convergence for Place_25000

Convergence Behavior on Ptest Netlists

For the Ptest netlists, best-known HPWL values are available for comparison. The convergence curves illustrate how the algorithm reduces the total HPWL from the initial placement toward a near-optimal solution.

The following trends are observed:

- The HPWL decreases rapidly in the early iterations due to high temperature and strong exploration.
- As the temperature decreases, improvements become smaller, indicating refinement of the solution.
- Smaller Ptest netlists converge faster compared to larger ones.
- Larger netlists show slower convergence due to the expanded search space.

The convergence plots for all Ptest netlists are presented below.

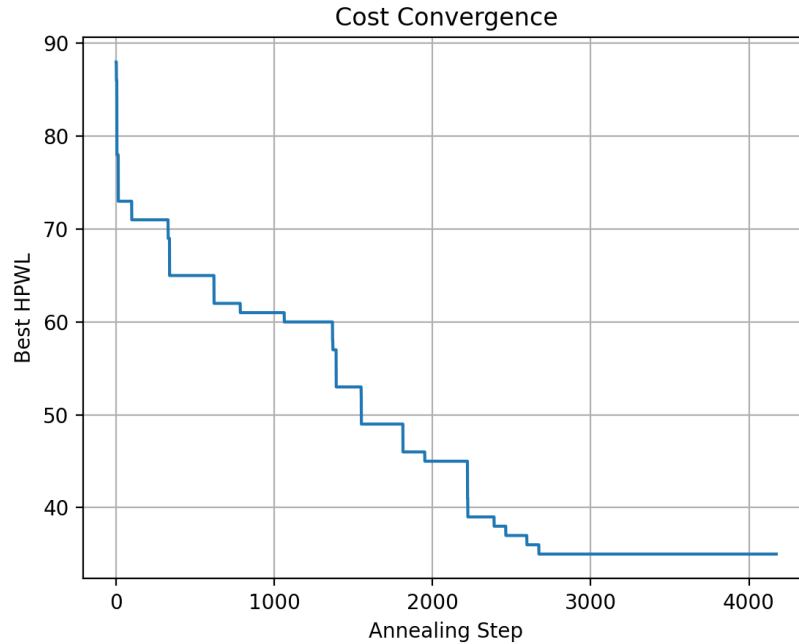


Figure 15: HPWL Convergence for Ptest_25

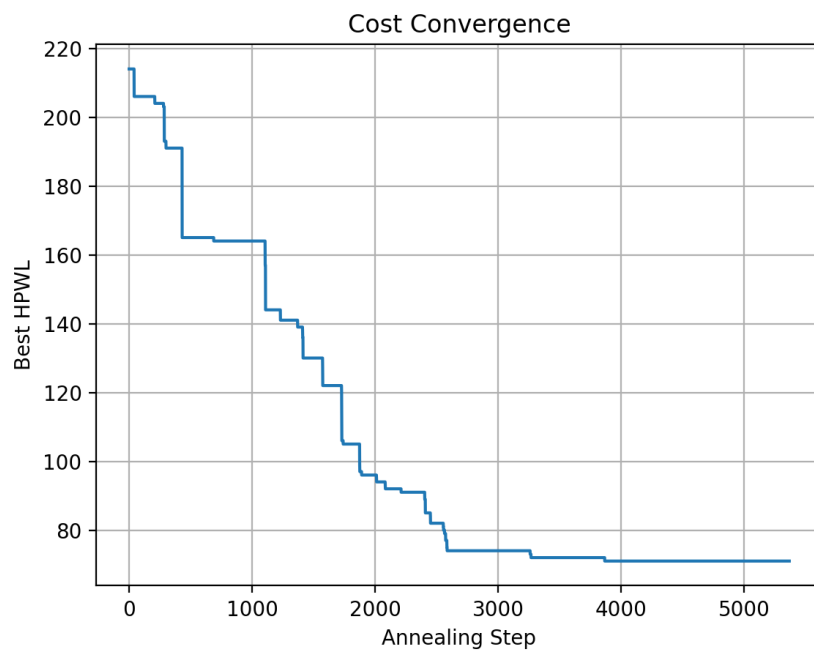


Figure 16: HPWL Convergence for Ptest_50

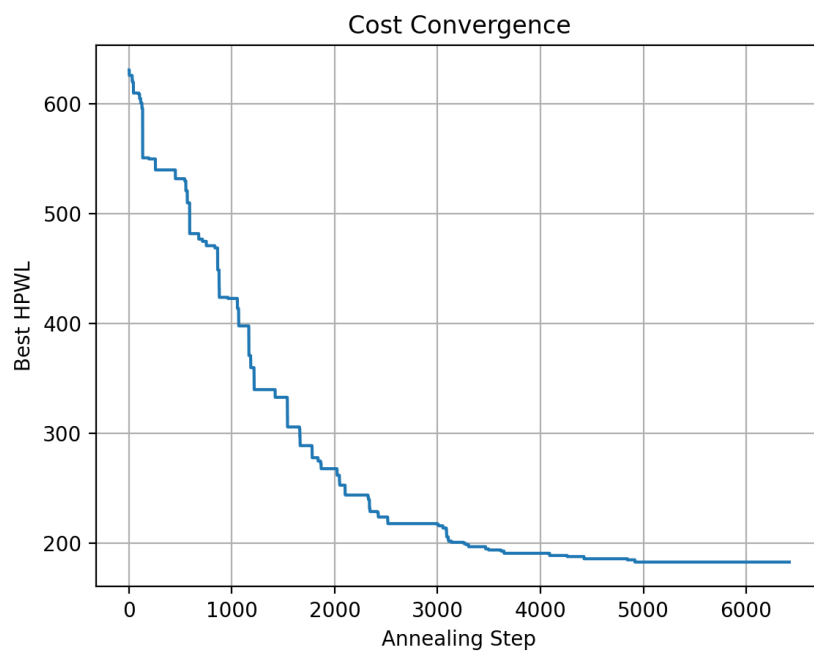


Figure 17: HPWL Convergence for Ptest_100

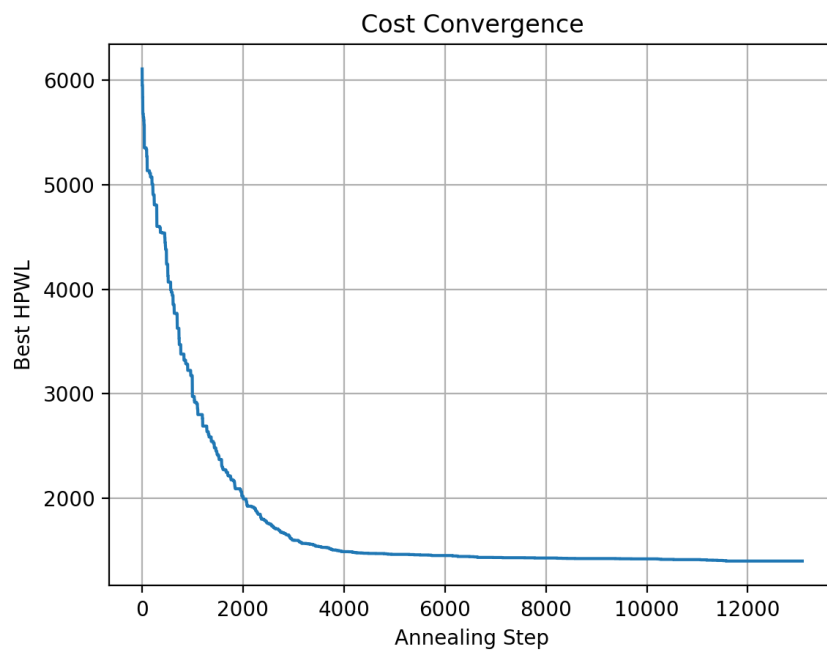


Figure 18: HPWL Convergence for Ptest_500

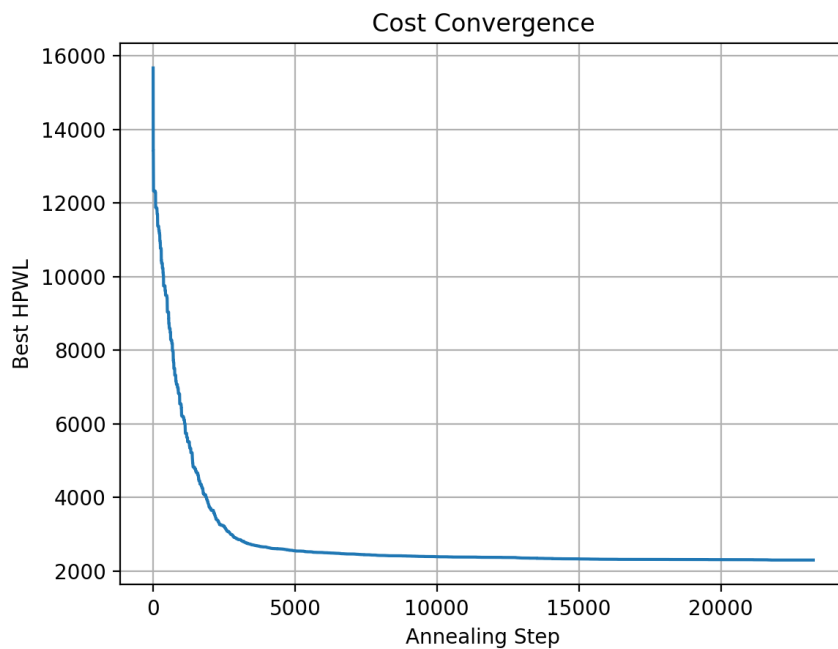


Figure 19: HPWL Convergence for Ptest_1000

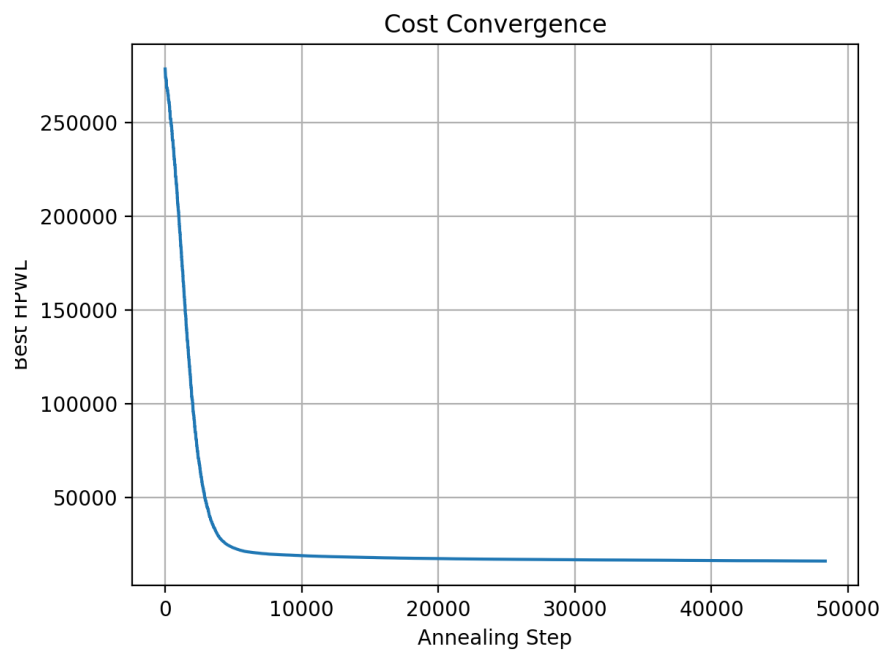


Figure 20: HPWL Convergence for Ptest_5000

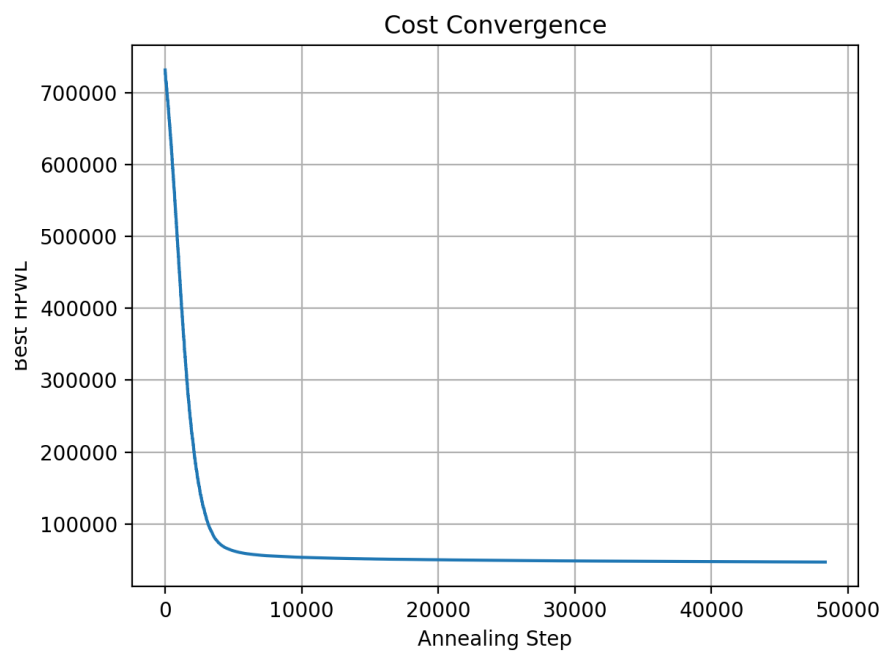


Figure 21: HPWL Convergence for Ptest_10000

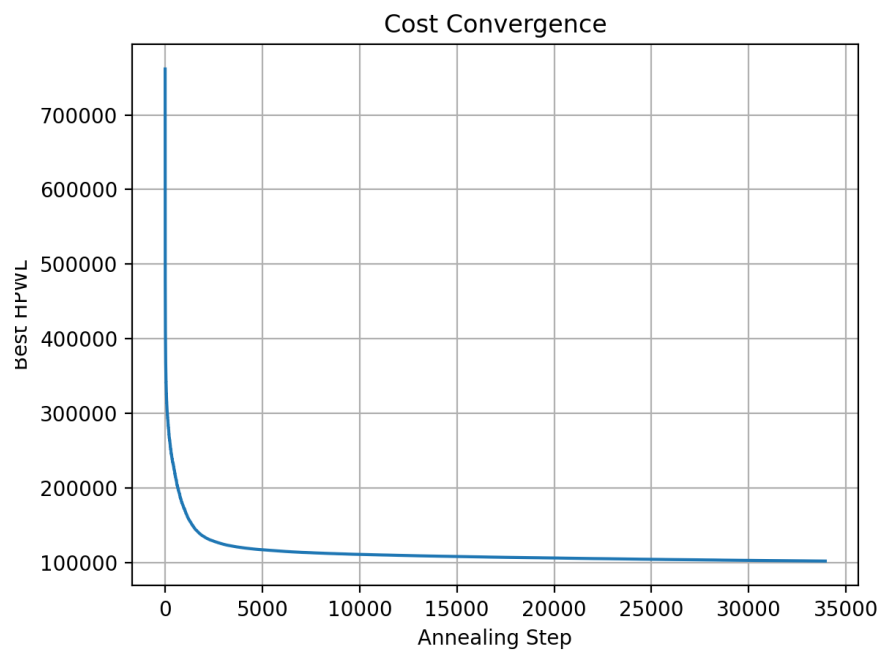


Figure 22: HPWL Convergence for Ptest_15000

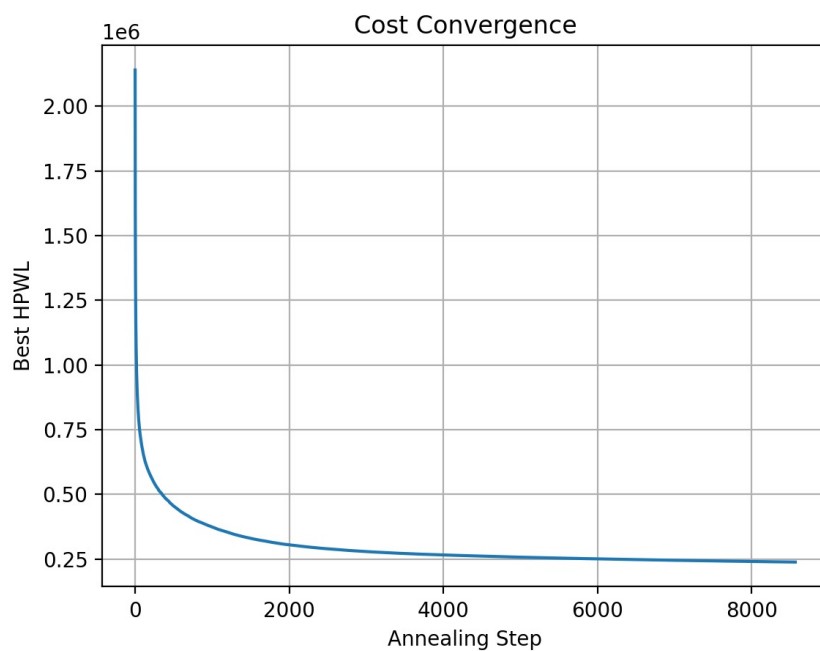


Figure 23: HPWL Convergence for Ptest_20000

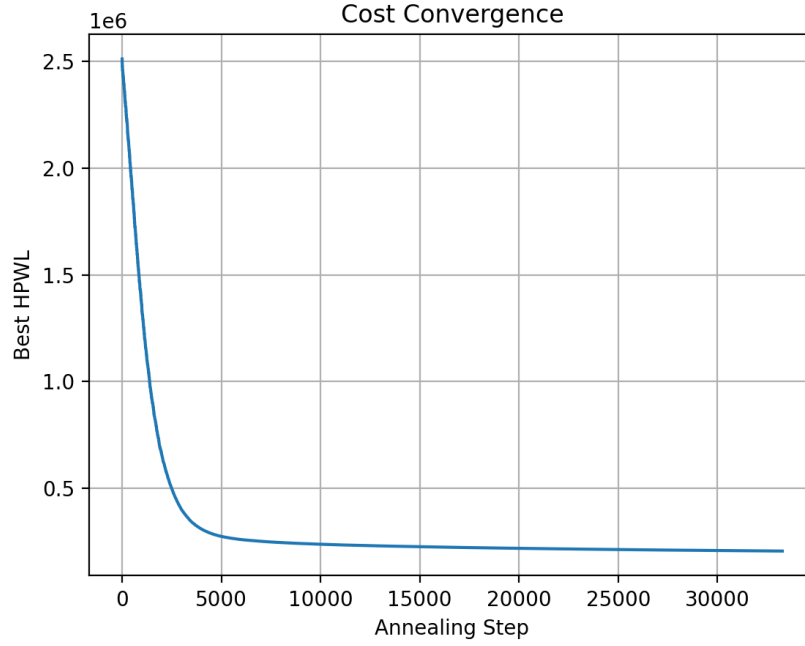


Figure 24: HPWL Convergence for Ptest_25000

Additional Evaluation and Observations

To better judge the quality of the algorithm, the software implementation, and the overall effort, several additional observations and experimental insights are worth noting.

Runtime Behavior and Scalability

The placer performs efficiently for small and medium netlists, but runtime increases significantly as the number of nets grows. For netlists larger than approximately 10k nets, the algorithm frequently reached the one-hour runtime limit before true convergence was achieved. This indicates that the implementation scales correctly but remains computationally intensive due to the large number of move evaluations required.

Convergence Characteristics

The cost convergence curves show a rapid reduction in wirelength during the early phase, followed by progressively smaller improvements. This behavior is consistent with simulated annealing: large improvements occur early, while later stages focus on refinement. For larger designs, convergence was still progressing when the time limit was reached, suggesting that additional runtime could further improve results.

Acceptance Behavior and Cooling Schedule

The acceptance curves indicate that move acceptance decreases smoothly as temperature decreases, confirming that the annealing schedule is functioning correctly. However, experiments showed that excessive early exploration (caused by high initial temperatures) consumed significant runtime without improving final solution quality under time constraints. A slightly more greedy early phase produced better results within the fixed runtime.

Solution Quality

The algorithm achieved substantial wirelength reduction compared to the initial placement, demonstrating effective global optimization capability. While the solutions may not match best-known benchmarks in all cases, the results fall within a reasonable range for a time-constrained heuristic approach.

Implementation Reliability

The tool was designed with incremental HPWL evaluation, stagnation detection, and structured logging, which improved efficiency and reproducibility. Logging of annealing metrics and automatic output generation enabled systematic evaluation of algorithm behavior across experiments.

Limitations

The primary limitations observed were:

- high computational cost for large netlists,
- sensitivity to temperature schedule parameters,
- premature termination due to runtime constraints rather than true convergence.

Effort and Engineering Considerations

Significant effort was invested in tuning parameters, analyzing annealing behavior, and instrumenting the algorithm to understand its performance. The addition of convergence tracking, acceptance analysis, and automated output generation helped ensure that results were reproducible and that algorithm behavior could be evaluated objectively.

Overall Assessment

Overall, the results indicate that the simulated annealing placer is effective and reliable within the imposed runtime constraints. The experimental analysis provides confidence in both the correctness of the implementation and the validity of the observed performance characteristics.

Retrospective

From our experiments, the most significant factor affecting performance was the temperature schedule and the amount of early exploration allowed by the simulated annealing process.

When we used a **high initial temperature**, the algorithm accepted many unfavorable moves. Although this enabled wide exploration of the solution space, in practice a large portion of the runtime was spent wandering randomly without producing meaningful improvement. Because the runtime was limited to one hour, excessive early exploration left insufficient time for refinement of the placement.

When we reduced the initial temperature and limited early exploration, the algorithm behaved more greedily in the early phase. Surprisingly, this improved the final results within the time limit. By restricting exploration time, the algorithm quickly moved toward lower-cost regions and spent more time refining the solution rather than randomly exploring.

This behavior suggests that, under strict time constraints, a more greedy early phase can outperform a fully exploratory schedule.

However, this does not necessarily mean the greedy schedule is globally optimal. If more runtime were available, maintaining a higher temperature for longer could allow the algorithm to escape deeper local minima and potentially converge to a better solution.

Another important observation is that for larger netlists (greater than $\sim 10k$ nets), the algorithm often did not truly converge. Instead, it was forced to terminate due to the one-hour runtime constraint. Given more time, we would allow the algorithm to remain at effective temperature ranges longer and observe its natural convergence behavior rather than stopping based solely on the time limit.

Future Improvements

If more time were available, we would:

- experiment with higher initial temperatures to evaluate long-term convergence quality,
- allow the algorithm to remain longer at mid-temperature ranges to observe true convergence,
- measure convergence rate instead of stopping strictly based on time,
- investigate adaptive cooling schedules to dynamically balance exploration and refinement.

Summary Insight

Our results suggest that under tight runtime constraints, limiting early exploration and encouraging faster refinement can produce better practical results. However, for larger problems or longer runtimes, a more exploratory schedule may lead to better global solutions.